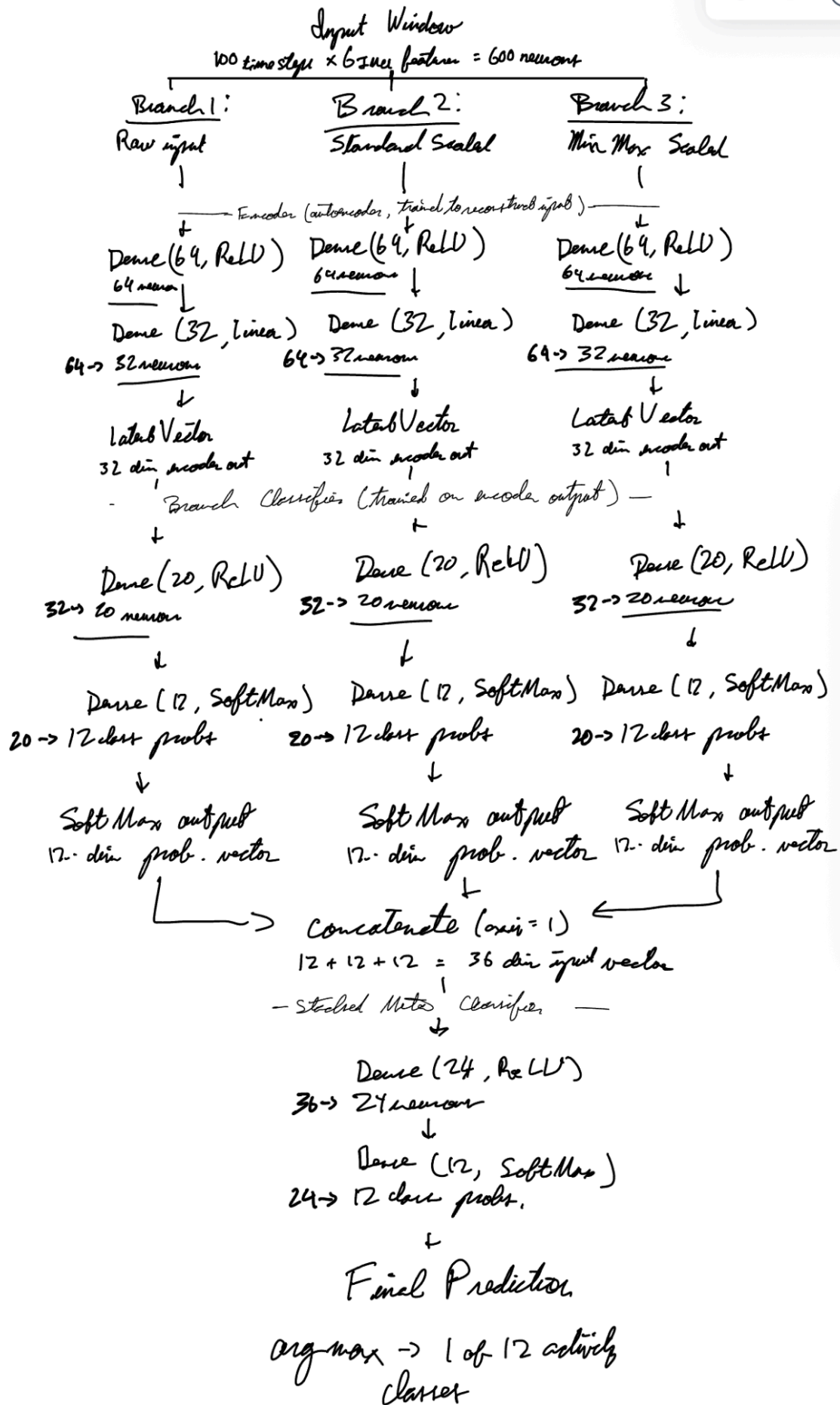


```
Timing: DSP 88 ms, inference 531 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  Idle: 0.882812
  Lift: 0.117188
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  Idle: 0.000000
  Lift: 0.996094
```

<https://studio.edgeimpulse.com/studio/1013725>

Question 1: Model architecture and ensemble flow



Question 2: Pruning and QAT methodology

Pruning: `prune_low_magnitude` with a `PolynomialDecay` schedule zeros out the smallest weights, ramping from 10% to 80% sparsity over 500 steps across 5 fine-tuning epochs. The `UpdatePruningStep` callback advances the schedule each batch.

Stripping and QAT: before applying quantization-aware training, the pruning wrappers have to be removed because the two toolkits can't be stacked on top of each other. Once stripped, QAT wraps the layers again but this time to simulate the rounding errors that will happen in int8, letting the model adjust its weights to that noise while still training in full precision.

Mask enforcement: needed because QAT's optimizer quietly un-does the pruning as gradient updates nudge zeroed-out weights back to small non-zero values. A custom callback re-zeros those weights after every batch to preserve the 80% sparsity that pruning achieved.

Int8 conversion: takes the trained model and converts all weights and activations from 32-bit floats to 8-bit integers. To do this correctly, the converter needs to see real input samples in the form of the representative dataset so it knows what range of values each layer actually produces and can choose the right scaling.

Why this matters for Arduino: the Nano 33 BLE Sense has very limited flash and RAM. Int8 weights take up significantly less space than floats, and integer math runs faster on that processor. Pruning cuts the number of actual multiplications needed significantly. Together they make it feasible to fit and run four separate models on a device with very limited memory resources.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

- A screenshot of the serial monitor output.

```
-----  
Final prediction: Lying down  
Class index: 2  
Confidence score: 0.3867  
-----
```

This output is pretty successful, because it classified the exact condition where the board's motion makes the most sense.

It guesses lying down every time, but the confidence score is never much more than 0.39. Of note is that other motions seem to have a lot more variability when it comes to prediction, a motion where I moved it in my hand couldn't pick between running, climbing stairs, and remaining lying down consistently.

Two things that could make deployment much more reliable would be to collect data with the board and its IMU, and perhaps a confidence threshold that prevents erroneous unconfident predictions under a certain threshold. Collecting the data that way would allow for the exact conditions the board moves in to be used for the inferences, which would reasonably be more reliable. The threshold would make overall outputs more "correct" by ignoring unconfident predictions, which would make the output more reliable.

Question 4: Deployment Behavior on the Arduino

After deployment, something that I noticed was that some of the movements seem to have overlapping characteristics, so when I'd intentionally try to move in such a way to elicit a specific prediction, it often couldn't choose between the correct one and other very similar classifications.

This could happen because the model only uses raw IMU values with no engineered features, so activities with similar motion profiles like walking vs. climbing stairs or jogging vs. running produce similar input windows. The decision boundaries between these classes are inherently close together, so small differences in movement, how the board was oriented, or minor scaling mismatches are enough to push a sample across the wrong boundary. The ensemble doesn't resolve this because all three branches see the same underlying signal and tend to be uncertain in the same direction.